Convolution

1. Convolutional layer can be thought of as a 2D analog of linear layer for 1D vectors, although there are some subtleties like weight sharing and sliding window approach for memory to be tractable in processing images.
    1. Note: in my opinion, resolution (width and height) of the image or feature maps are of secondary importance compared to the feature dimension of the feature maps since we're able to achieve the desired resolution of almost any sizes using for example Spatial Pooling Pyramid or even using zero padding.
2. Convolution formula $W_{out} = \frac{W_{in}+2P+K}{stride} + 1$.
3. Tip: By padding the input image with $K//2$ the output will be of the same width and height with the input. example:
    1. padded image size = 10 + 3//2 + 3//2 = 12
    2. convolve with kernel of size K = 3.
    3. get original size back: $W_{out} = \frac{12+0-3}{1} + 1 = 9 + 1 = 10$
4. Tip: By padding the output with $K-1$, and convolving it with the kernel will return an feature map with the same size as the original input. This property is used in conv backprop, example:
    1. Original input size: $W_{in} = 10$
    2. $kernel\_size = 3$
    3. $W_{out} = \frac{10+0-3}{1} + 1 = 8$
    4. now pad output with $K-1$,
    5. $W_{out\_padded} = 8 + 2 + 2 = 12$
    6. convolve resulting output with kernel size of K:
    7. $\frac{12+0-3}{1} + 1 = 10$.
    8. Since in backpropagation, $dLdx = dLdz \circledast flipped(W)$, where W is the filter.
5. How big is our receptive field?
    1. $r_0 = \sum_{l=1}^{L} \left((k_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$
        1. r_0 is receptive field size
        2. k_l is kernel size at layer l.
        3. $\sum_{l=1}^{L}$ sum over network layers.
        4. $\prod_{i=1}^{l-1}$ is the product of strides up to layer l.
6. conv1d

```python
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 13 17:53:49 2026

@author: reina
"""

import numpy as np
from resampling1d import *


def f1(C_out, C_in, K):
    return np.random.randint(0, 5, size=(C_out, C_in, K))


def f2(C_out):
    return np.random.randint(0, 3, size=(C_out, 1))


class Conv1d_stride1:
    def __init__(
        self, in_channels, out_channels, kernel_size, weight_init_fn, bias_init_fn
    ):
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size

        if weight_init_fn is None:
            self.W = np.random.normal(0, 1.0, (out_channels, in_channels, kernel_size))
        else:
            self.W = weight_init_fn(out_channels, in_channels, kernel_size)
        if bias_init_fn is None:
            self.b = np.zeros(shape=(out_channels, 1))
        else:
            self.b = bias_init_fn(out_channels)
        self.dLdW = None
        self.dLdb = None

    def forward(self, x):
```

```
        self.x = x  # save input for backprop
        N, C_in, W_in = x.shape
        W_out = (W_in - self.K) + 1
        z = np.empty(shape=(N, self.C_out, W_out), dtype=np.float64)
        for i in range(W_out):
            z[:, :, i] = np.tensordot(
                x[:, :, i : i + self.K], self.W, axes=((1, 2), (1, 2))
            ) + self.b.reshape(-1)
        return z

    def backward(self, dLdz):
        N, C_out, W_out = dLdz.shape
        N, C_in, W_in = self.x.shape
        assert C_out == self.C_out and C_in == self.C_in
        # find dLdW
        self.dLdW = np.empty(shape=(C_out, C_in, self.K), dtype=np.float64)
        # for i in range(C_out):
        #     # Convolve input x with dLdz as filter.
        #     for j in range(self.K):
        #         # With tensordot, broadcasting is handled implicitly.
        #         # Size of immediate result after convolution is (C_in, 1) reshape to (C_in,).
        #         self.dLdW[i, :, j] = np.tensordot(self.x[:, :, j:j+W_out], dLdz[:, i:i+1, :],
        #                                           axes=((0,2),(0,2))).reshape(-1)
        for i in range(self.K):
            self.dLdW[:, :, i] = np.tensordot(
                self.x[:, :, i : i + W_out], dLdz, axes=((0, 2), (0, 2))
            ).transpose(1, 0)

        # find dLdb
        self.dLdb = np.sum(dLdz, axis=(0, 2))
        # find dLdx
        dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)
        # pad dLdz with K-1 zeros left and right
        # dLdz size: before: W_in-K+1, after: W_in-K+1+2(K-1) = W_in+K-1
        # i.e. to get back original input size, W_in, after convolving dLdz with 180^o rotated kernel as filter.
        dLdz = np.pad(
            dLdz, pad_width=((0, 0), (0, 0), (self.K - 1, self.K - 1)), mode="constant"
        )

        # for i in range(C_out):
        #     # Convolve dLdz with 180^o rotated kernel as filter.
        #     for j in range(W_in):
        #         # because all outputs are of the same size (N, C_in) we should accumulate the results.
        #         dLdx[:, :, j] += np.tensordot(dLdz[:, i:i+1, j:j+self.K], np.flip(self.W[i:i+1, :, :],
        #                                                                           axis=2), axes=((1,2),(0,2)))

        for i in range(W_in):
            dLdx[:, :, i] = np.tensordot(
                dLdz[:, :, i : i + self.K],
                np.flip(self.W, axis=2),
                axes=((1, 2), (0, 2)),
            )
        return dLdx


class Conv1d:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size,
        stride,
        padding=0,
        weight_init_fn=None,
        bias_init_fn=None,
    ):

        self.stride = stride
        self.padding = padding
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size
        self.conv1d_stride1 = Conv1d_stride1(
            self.C_in, self.C_out, self.K, weight_init_fn, bias_init_fn
        )
        self.downsampled1d = Downsample1d(downsampling_factor=stride)

    def forward(self, x):
        # pad with zeros
        x = np.pad(
            x, pad_width=((0, 0), (0, 0), (self.padding, self.padding)), mode="constant"
        )
        # Conv1d forward
        z = self.conv1d_stride1.forward(x)
        # Downsample1d forward
        z = self.downsampled1d.forward(z)
        return z

    def backward(self, dLdz):
        # Downsample1d backward
```

```
        dLdx = self.downsampled1d.backward(dLdz)
        # Conv1d backward
        dLdx = self.conv1d_stride1.backward(dLdx)
        # unpad zeros
        if self.padding > 0:
            dLdx = dLdx[:, :, self.padding : -self.padding]
        return dLdx


if __name__ == "__main__":
    N = 10
    C_in = 5
    C_out = 8
    K = 3
    W_in = 5

    conv1d_stride1 = Conv1d_stride1(C_in, C_out, K, f1, f2)
    conv1d = Conv1d(
        C_in, C_out, K, stride=2, padding=0, weight_init_fn=f1, bias_init_fn=f2
    )

    # forward
    x = np.random.randint(0, 10, size=(N, C_in, W_in))
    z = conv1d.forward(x)

    # backward
    dLdz = np.random.random(z.shape)
    dLdx = conv1d.backward(dLdz)

    # print(h)
    print("x.shape", x.shape)
    print("z.shape ", z.shape)
    print("dLdx.shape ", dLdx.shape)
    print("dLdZ.shape", dLdz.shape)
```

7. conv2d

```
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 13 18:03:49 2026

@author: reina
"""

import numpy as np
from resampling2d import *


def f1(C_out, C_in, K1, K2):
    return np.random.randint(0, 5, size=(C_out, C_in, K1, K2))


def f2(C_out):
    return np.random.randint(0, 3, size=(C_out, 1))


class Conv2d_stride1:
    def __init__(
        self, in_channels, out_channels, kernel_size, weight_init_fn, bias_init_fn
    ):
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size

        self.W = weight_init_fn(out_channels, in_channels, kernel_size, kernel_size)
        self.b = bias_init_fn(out_channels)

        self.dLdW = None
        self.dLdb = None

    def forward(self, x):
        self.x = x
        N, C_in, H_in, W_in = x.shape
        H_out = H_in - self.K + 1
        W_out = W_in - self.K + 1
        z = np.empty(shape=(N, self.C_out, H_out, W_out), dtype=np.float64)
        for i in range(H_out):
            for j in range(W_out):
                z[:, :, i, j] = np.tensordot(
                    x[:, :, i : i + self.K, j : j + self.K],
                    self.W,
                    axes=((1, 2, 3), (1, 2, 3)),
                ) + self.b.reshape(-1)
        return z

    def backward(self, dLdz):
        N, C_out, H_out, W_out = dLdz.shape
        N, C_in, H_in, W_in = self.x.shape
        assert C_out == self.C_out and C_in == self.C_in
        # find dLdW
        self.dLdW = np.empty(shape=(C_out, C_in, self.K, self.K), dtype=np.float64)
```

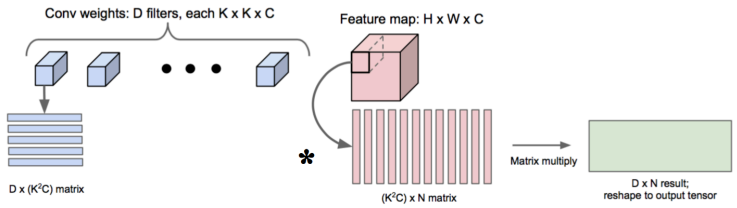
```
        # for i in range(C_out):
        #     for j in range(self.K):
        #         for k in range(self.K):
        #             self.dLdW[i, :, j, k] = np.tensordot(self.x[:, :, j:j+H_out, k:k+W_out],
        #                                                  (dLdz[:, i, :, :])[:, np.newaxis, :, :],
        #                                                  axes=((0,2,3),(0,2,3))).reshape(-1)
        for i in range(self.K):
            for j in range(self.K):
                self.dLdW[:, :, i, j] = np.tensordot(
                    self.x[:, :, i : i + H_out, j : j + W_out],
                    dLdz,
                    axes=((0, 2, 3), (0, 2, 3)),
                ).transpose(1, 0)

        # find dLdb
        self.dLdb = np.sum(dLdz, axis=(0, 2, 3))
        # find dLdx
        dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)
        dLdz = np.pad(
            dLdz,
            pad_width=(
                (0, 0),
                (0, 0),
                (self.K - 1, self.K - 1),
                (self.K - 1, self.K - 1),
            ),
            mode="constant",
        )
        # for i in range(C_out):
        #     for j in range(H_in):
        #         for k in range(W_in):
        #             dLdx[:, :, j, k] += np.tensordot((dLdz[:, i, j:j+self.K, k:k+self.K])[:, np.newaxis, :, :],
        #                                              np.flip((self.W[i, :, :, :])[np.newaxis, :, :, :],
        #                                              axis=(2,3)), axes=((1,2,3),(0,2,3)))
        for i in range(H_in):
            for j in range(W_in):
                dLdx[:, :, i, j] = np.tensordot(
                    dLdz[:, :, i : i + self.K, j : j + self.K],
                    np.flip(self.W, axis=(2, 3)),
                    axes=((1, 2, 3), (0, 2, 3)),
                )
        return dLdx


class Conv2d:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size,
        stride,
        padding=0,
        weight_init_fn=None,
        bias_init_fn=None,
    ):
        self.stride = stride
        self.padding = padding
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size
        self.conv2d_stride1 = Conv2d_stride1(
            self.C_in, self.C_out, self.K, weight_init_fn, bias_init_fn
        )
        self.downsample2d = Downsample2d(downsampling_factor=stride)

    def forward(self, x):
        # pad with zeros
        x = np.pad(
            x,
            pad_width=(
                (0, 0),
                (0, 0),
                (self.padding, self.padding),
                (self.padding, self.padding),
            ),
            mode="constant",
        )
        # Conv2d forward
        z = self.conv2d_stride1.forward(x)
        # Downsample forward
        z = self.downsample2d.forward(z)
        return z

    def backward(self, dLdz):
        # Downsample backward
        dLdx = self.downsample2d.backward(dLdz)
        # Conv2d backward
        dLdx = self.conv2d_stride1.backward(dLdx)
        # unpad zeros
        if self.padding > 0:
```

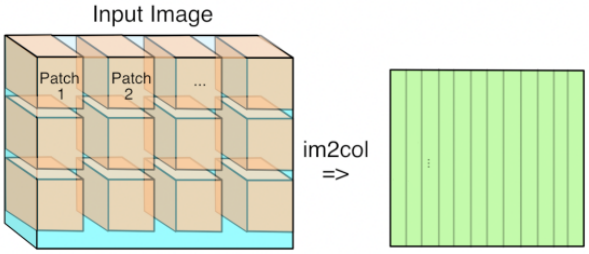
```
            dLdx = dLdx[
                :, :, self.padding : -self.padding, self.padding : -self.padding
            ]
        return dLdx


if __name__ == "__main__":
    # for testing purposes
    N = 10
    C_in = 5
    C_out = 8
    K = 3
    H_in = 5
    W_in = 5

    conv2d_stride1 = Conv2d_stride1(C_in, C_out, K, f1, f2)
    conv2d = Conv2d(C_in, C_out, K, 2, 0, f1, f2)

    # forward
    x = np.random.randint(0, 10, (N, C_in, H_in, W_in))
    z = conv2d_stride1.forward(x)
    # backward
    dLdz = np.random.randint(0, 10, size=z.shape)
    dLdx = conv2d_stride1.backward(dLdz)

    print("x.shape ", x.shape)
    print("z.shape ", z.shape)

    # print("dLdx ", dLdx)
    print("dLdx.shape ", dLdx.shape)
    print("dLdz.shape ", dLdz.shape)

    # forward
    x = np.random.randint(0, 10, (N, C_in, H_in, W_in))
    z1 = conv2d.forward(x)
    # backward
    dLdz1 = np.random.randint(0, 10, size=z1.shape)
    dLdx1 = conv2d.backward(dLdz1)

    print("x.shape ", x.shape)
    print("z1.shape ", z1.shape)
    print("dLdx1.shape ", dLdx1.shape)
    print("dLdz1.shape ", dLdz1.shape)
```

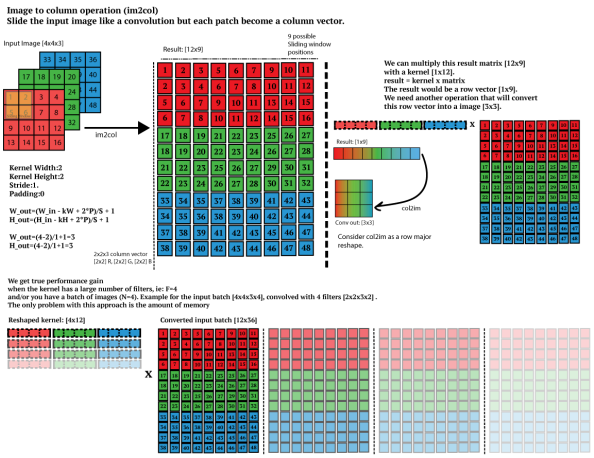
8. im2col
    1. As shown on previous source code, if we do not use `np.tensordot` we will use a lot of for-loops to implement the convolutions, an alternative to this is using im2col. On this section we will learn how to implement convolutions on a vectorized fashion.
    2. If we expand all possible windows on memory and perform the dot product as a matrix multiplication, we'll get 200x or more speedups, at the expense of more memory consumption.
    3. For example, if the input is [227x227x3] and it is to be convolved with 11x11x3 filters at stride 4 and padding 0, then we would take [11x11x3] blocks of pixels in the input and stretch each block into a column vector of size $11*11*3 = 363$.
    4. Calculating with input 227 with stride 4 and padding 0, gives ((227-11)/4)+1 = 55 locations along both width and height, leading to an output matrix X_{col} of size [363 x 3025], Note: 55 * 55 = 3025.
    5. Here every column is a stretched out receptive field (patch with depth) and there are $55 * 55 = 3025$ of them in total.
    6. The weights of the CONV layer are similarly stretched out into rows. For example, if there are 96 filters of size [11x11x3] this would give a matrix W_{row} of size [96 x 363], where 11x11x3=363
    7. After the image and the kernel are converted, the convolution can be implemented as a simple matrix multiplication, in our case it will be W_{col} [96 x 363] multiplied by X_{col} [363 x 3025] resulting as a matrix [96 x 3025], that need to be reshaped back to [55x55x96].
    8. This final reshape can also be implemented as a function called col2im.
    9. Notice that some implementations of im2col will have this result **transposed**, if this is the case then the order of the matrix multiplication must be changed.
9. To summarize how we calculate the im2col output sizes:

```
[img_height, img_width, img_channels] = size(img);
newImgHeight = floor(((img_height + 2*P - ksize) / S)+1);
newImgWidth = floor(((img_width + 2*P - ksize) / S)+1);
cols = single(zeros((img_channels*ksize*ksize),(newImgHeight * newImgWidth)));
```

10. im2col (transposed version)
    1. Reshape image to be convolved to shape $(H_{out} * W_{out}, K^2C)$, and reshape kernel to shape (K^2C_{in}, C_{out}).
    2. $(H_{out} * W_{out}, K^2C_{in}) \cdot (K^2C_{in}, C_{out}) = (H_{out} * W_{out}, C_{out})$
    3. In standard convolution we aggregate across filter size, K, and channels, C. Hence, why reshaping the image tensor and the filters into of size K^2C in one of its two dimensions.



11. 

12. 

References:

1. https://velog.io/@tobigs1516_image/FCN-Fully-Convolution-Network-for-semantic-segmentation-review
2. https://leonardoaraujosantos.gitbook.io/artificial-inteligence/machine_learning/deep_learning/image_segmentation